NotesApp Transit Audit

Pair: Kexin Guo and Hongrui Bian

We audited login traffic in our own NotesApp instance before and after enabling HTTPS. HTTP exposed the login credentials in plaintext. HTTPS encrypted the captured application traffic, while the self-signed certificate still triggered a browser trust warning.

Finding 1 Login over plain HTTP

Exposed: The username and password were transmitted in plaintext. The captured HTTP request body contained username=alice&password=alicepw.

Who can read it: Anyone able to capture the relevant unencrypted network traffic.

Severity: High, because an attacker who captures the login credentials could use them to access the account.

Fix: Configure an Nginx reverse proxy with HTTPS/TLS and remove the direct host port mapping to Flask, keeping the web service accessible through the internal Docker network.

Evidence: Figure 1 shows the HTTP POST /login request with readable credentials.

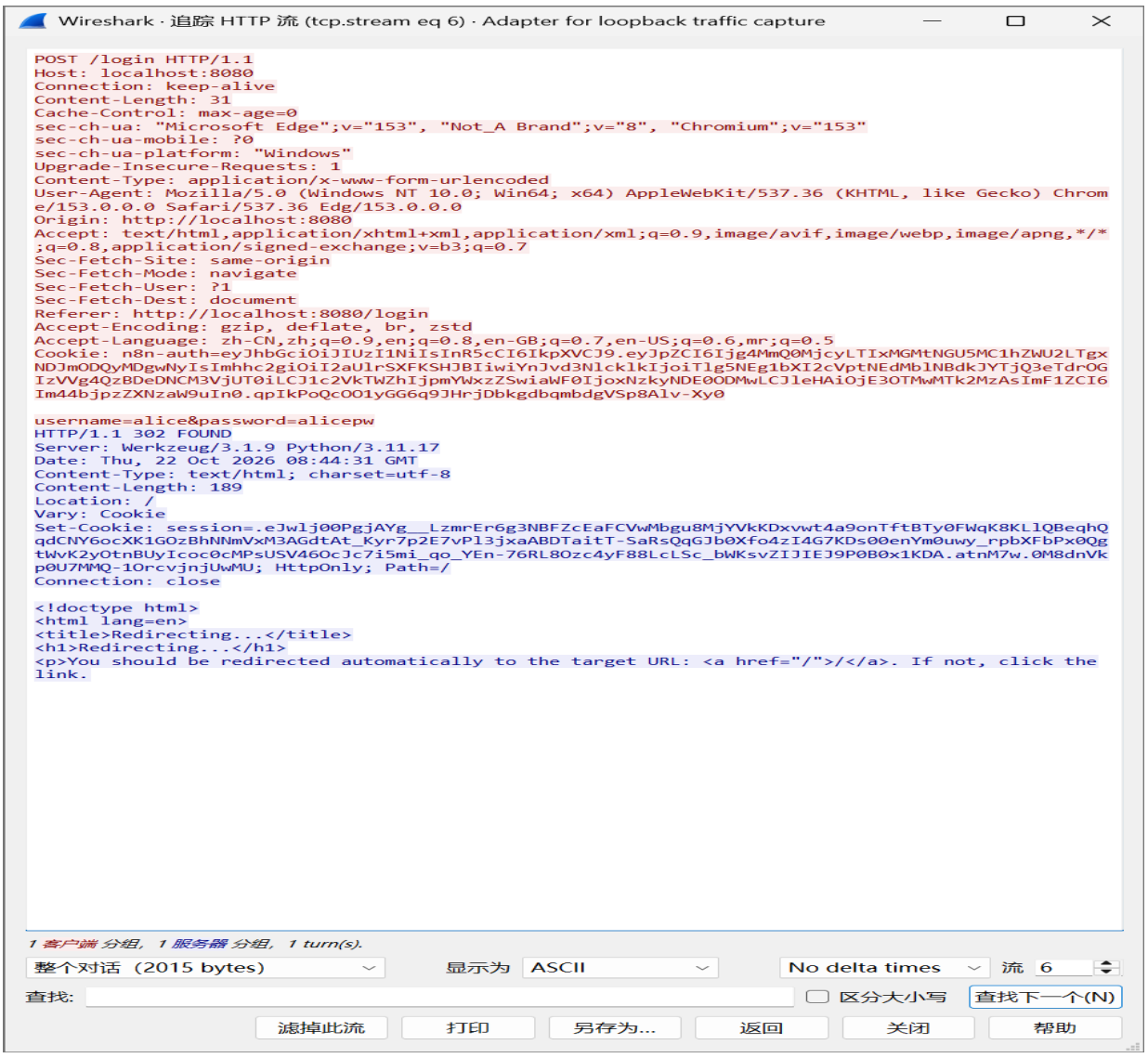


Figure 1. Plain HTTP exposes the login username and password.

Fix verification Login over HTTPS

Password readable: No.

After enabling HTTPS, we captured the login traffic again. Wireshark showed TLS 1.3 and encrypted Application Data. Searching the captured TCP stream for alicepw returned no match.

The web service no longer published port 8080 to the host. Nginx published ports 80 and 443, and its configuration redirected HTTP requests to HTTPS.

Evidence: Figures 2 and 3 show TLS traffic on port 443 and the encrypted TCP stream.

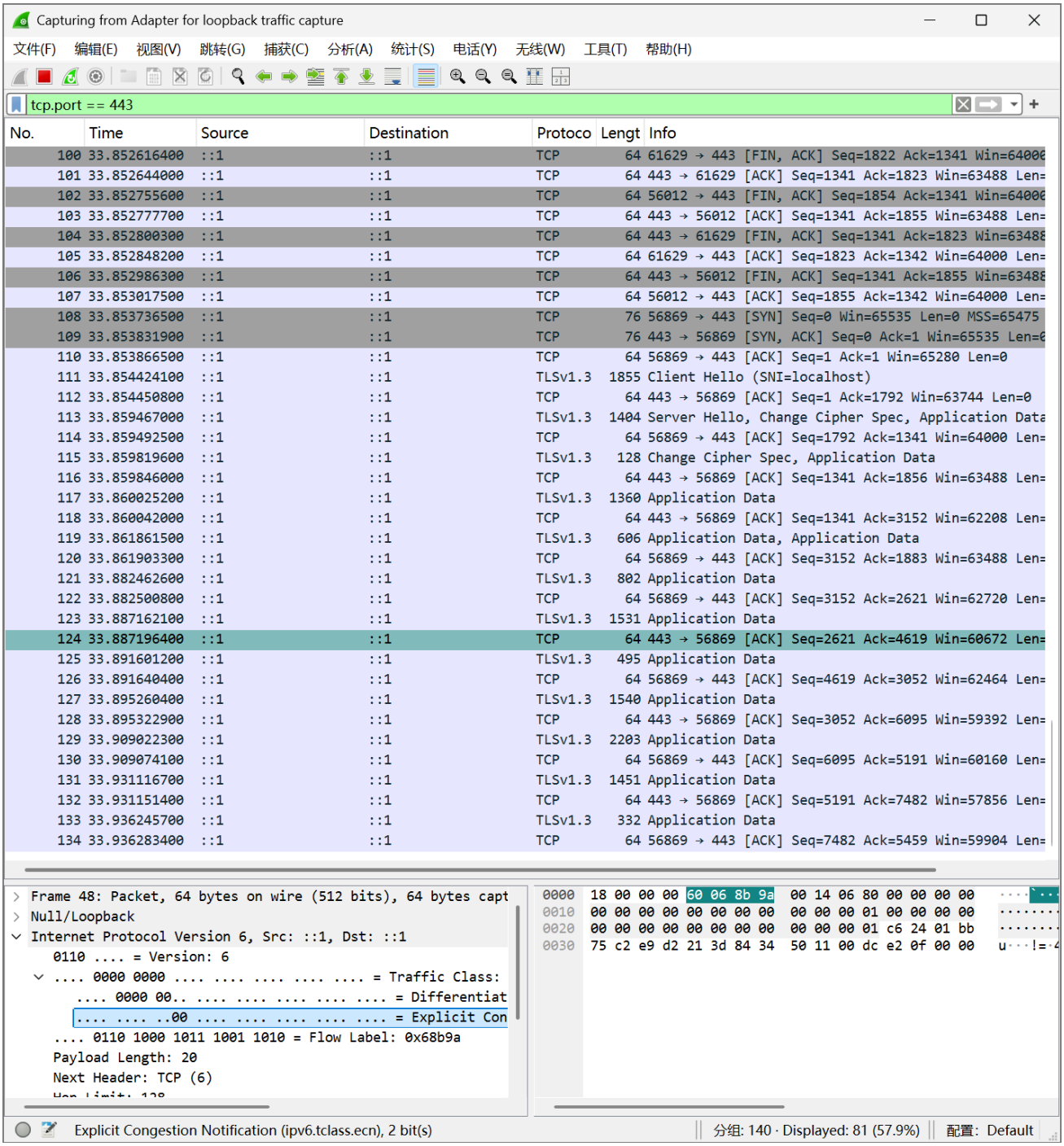


Figure 2. HTTPS traffic is carried as TLS 1.3 Application Data.

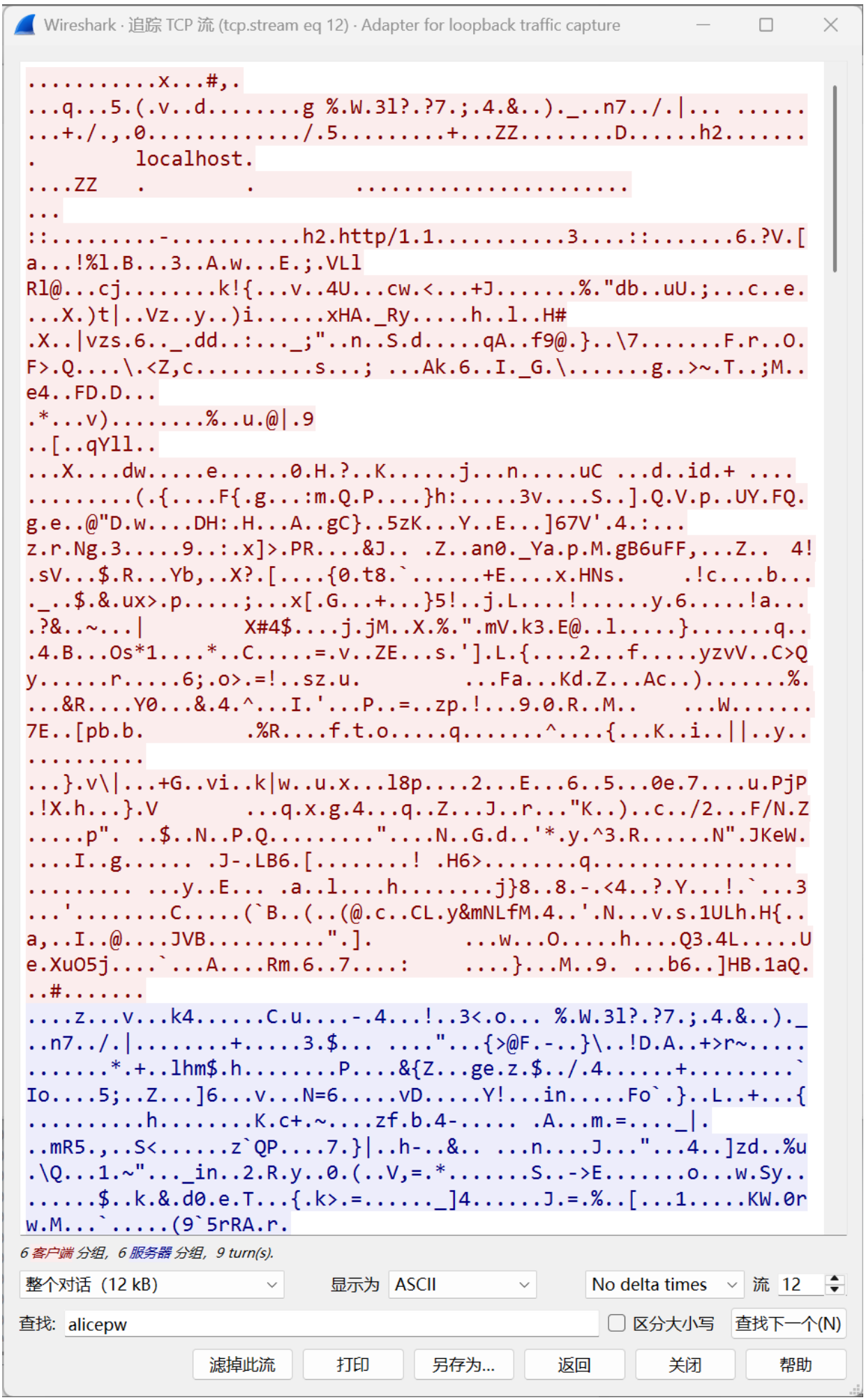

Wireshark · 追踪 TCP 流 (tcp.stream eq 12) · Adapter for loopback traffic capture
..........x...#,.
...q...5.(.v..d.......g %.W.3l?.?7.;.4.&..)._..n7../.|... .....
...+./.,.0.............../.5.........+...ZZ........D......h2.....
.   localhost.
....ZZ   .   .   ..
...
::.........-..........h2.http/1.1..........3....::.......6.?V.[
a...!%l.B...3..A.w...E.;.VLl
Rl@...cj........k!{...v..4U...cw.<...+J.......%."db..uU.;...c..e.
...X.)t|..Vz..y..)i......xHA._Ry.....h..l..H#
.X..|vzs.6.._.dd..:..._;"..n..S.d.....qA..f9@.}..\7.......F.r..O.
F>.Q....\.<Z,c..........s...; ...Ak.6..I._G.\.......g..>~.T..;M..
e4..FD.D...
.*...v).........%..u.@|.9
..[..qYll..
...X....dw.....e......0.H.?..K......j...n.....uC ...d..id.+ ....
.........(.{....F{.g...:m.Q.P....}h:.....3v....S..].Q.V.p..UY.FQ.
g.e..@"D.w....DH:.H...A..gC}..5zK...Y..E...]67V'.4.:...
z.r.Ng.3.....9..:.x]>.PR....&J.. .Z..an0._Ya.p.M.gB6uFF,...Z.. 4!
.sV...$.R...Yb,..X?.[....{0.t8.`......+E....x.HNs.   .!c....b...
._..$.&.ux>.p.....;...x[.G...+...}5!..j.L....!......y.6.....!a...
.?&..~...| X#4$....j.jM..X.%.".mV.k3.E@..l.....}.......q..
.4.B...Os*1....*..C.....=.v..ZE...s.'].L.{....2...f.....yzvV..C>Q
y......r.....6;.o>.=!..sz.u.   ...Fa...Kd.Z...Ac..).......%.
...&R....Y0...&.4.^...I.'...P..=..zp.!...9.0.R..M..   ...W.......
7E..[pb.b.   .%R....f.t.o.....q.......^....{...K..i..||..y..
....
...}.v\|...+G..vi..k|w..u.x...l8p....2...E...6..5...0e.7....u.PjP
.!X.h...}.V   ...q.x.g.4...q..Z...J..r..."K..)..c../2...F/N.Z
.....p". ..$..N..P.Q........."....N..G.d..'*.y.^3.R......N".JKeW.
....I..g..... .J-.LB6.[........! .H6>........q.................
....y..E... .a..l....h........j}8..8.-.<4..?.Y...!.`...3
...'........C.....(`B..(..(@.c..CL.y&mNLfM.4..'.N...v.s.1ULh.H{..
a,..I..@....JVB.........".].   ...w...O.....h....Q3.4L.....U
e.XuO5j....`...A....Rm.6..7....:   ....}...M..9. ...b6..]HB.1aQ.
..#.......
....z...v...k4......C.u....-.4...!..3<.o... %.W.3l?.?7.;.4.&..)._
..n7../.|........+.....3.$... ...."...{>@F.-..}\..!D.A..+>r~.....
.......*.+..lhm$.h........P....&{Z...ge.z.$../.4......+.........`
Io....5;..Z...]6...v...N=6.....vD.....Y!...in.....Fo`.}..L..+...{
..........h........K.c+.~....zf.b.4-..... .A...m.=...._|.
..mR5.,..S<......z`QP....7.}|..h-..&.. ...n....J..."...4..]zd..%u
.\Q...1.~"..._in..2.R.y..0.(..V,=.*.......S..->E.......o...w.Sy..
......$..k.&.d0.e.T...{.k>.=......_]4.....J.=.%..[...1.....KW.0r
w.M...`.....(9`5rRA.r.
6 客户端 分组，6 服务器 分组，9 turn(s).
整个对话（12 kB）
显示为
ASCII
No delta times
流
12
查找:
alicepw
区分大小写
查找下一个(N)
滤掉此流
打印
另存为...
返回
关闭
帮助

Figure 3. The captured HTTPS TCP stream does not expose the plaintext password.

Finding 2 Certificate trust

GitHub certificate issuer: Sectigo Public Server Authentication CA DV E36 (Sectigo Limited).



Figure 4. GitHub certificate subject and issuer.

Local certificate issuer: localhost. The certificate is self-signed.

Missing trust: Our certificate does not have a certificate chain leading to a root certificate authority trusted by the browser. The browser displayed NET::ERR_CERT_AUTHORITY_INVALID.



Figure 5. The localhost certificate is self-signed and is not trusted by the browser.

Verdict: Encryption and authenticated server identity are different properties. Our self-signed certificate allows encrypted communication, but the browser cannot establish trust in the server identity through its trusted certificate authorities. A valid certificate for the requested hostname, with a trusted certificate chain, provides that authentication.

Final state

NotesApp is served over HTTPS through Nginx using a self-signed certificate for this lab. Flask's direct host port mapping has been removed. The captured HTTPS traffic no longer exposes the login credentials in plaintext. The certificate trust warning remains because the certificate is self-signed.